

Технический отчёт по проекту локального индикатора утомления и ассистента восстановления работников

Статус документа: актуальное состояние проекта на 11 августа 2026 года.

Текущая сборка: Step 7 — `fatigue_assistant_step7_state_ui`.

Рабочее название интерфейса: «Ассистент восстановления».

Назначение документа: передать проект новому программисту, исследователю или другому ИИ так, чтобы он мог продолжить разработку без необходимости заново восстанавливать архитектуру, продуктовые решения, историю ошибок и будущий план.

Главное, что новый разработчик должен понять с самого начала: проект **не является медицинским диагностическим прибором и на текущем этапе ещё не доказал способность точно измерять физиологическую усталость человека**. Его конечная задача — локально и ненавязчиво объединять несколько независимых признаков утомления, сравнивать их с индивидуальной нормой конкретного сотрудника, определять устойчивость изменений и в подходящий момент предлагать наиболее уместный способ восстановления. Это согласуется с подходом NIOSH: рабочее утомление многофакторно, может быть связано со сном, длительностью работы, стрессом, психической нагрузкой и характером задачи; универсального единственного показателя или одного решения для всех ситуаций нет. ¹

Концепция продукта, аудитория и обязательные принципы

Первоначальная бизнес-задача формулировалась применительно к **сотрудникам Т-Банка и Сбербанка**, прежде всего к людям, которые проводят значительную часть рабочего дня за компьютером: работают с документами, таблицами, внутренними системами, аналитикой, кодом, клиентскими обращениями, операционными интерфейсами и другими экранными задачами. Это пока направление продукта, а не утверждение о наличии контракта или пилота с указанными банками.

Основная цель:

снизить накопление утомления в течение рабочего дня и тем самым поддерживать качество, устойчивость и эффективность работы за счёт своевременного и персонализированного восстановления.

Важно, что продукт не должен превращаться в инструмент усиления рабочего давления. Его смысл не в том, чтобы выжать из человека больше работы, а в том, чтобы уменьшить случаи, когда сотрудник часами продолжает работать в ухудшившемся состоянии, хотя короткая своевременная разгрузка могла бы восстановить часть работоспособности. NIOSH связывает рабочее утомление со снижением внимания и концентрации, замедлением реакции, ухудшением кратковременной памяти и суждения; время на задаче, особенно монотонной, также увеличивает вероятность утомления. ²

Как должна работать готовая программа

Это **фоновая локальная программа на рабочем компьютере сотрудника**.

Она не должна требовать, чтобы человек держал открытым специальное окно, смотрел на видео с веб-камеры или нажимал клавишу **P**, чтобы сообщить о перерыве. Именно по этой причине ранний механизм ручного перерыва через клавишу **P** был признан непригодным: он работал только тогда, когда диагностическое окно OpenCV находилось в фокусе.

В производственной версии предполагается:

Ситуация	Поведение программы
Сотрудник работает	мониторинг идёт в фоне
Сотрудник читает и не трогает мышь	работа продолжается, это не считается отсутствием или усталостью
Сотрудник отошёл	оценка утомления приостанавливается
Экран заблокирован	оценка должна приостанавливаться
Сотрудник вернулся	действует защитный период RETURNING
Нужен перерыв	появляется компактное уведомление
Нужна самооценка	открывается небольшое окно с двумя шкалами
Пользователь хочет разобраться	раскрывается Подробнее
Пользователь хочет проверить камеру	отдельно открывается диагностическое превью
Пользователь не хочет мониторинг	должен иметь возможность явно приостановить его

Готовый продукт в идеале должен запускаться вместе с рабочей сессией Windows и большую часть времени находиться в системном трее. **Step 7 пока является оконным прототипом**, а не окончательным tray-приложением.

Что именно программа должна определять

Нужно строго различать четыре понятия:

Рабочий контекст отвечает на вопрос:

Что человек сейчас делает относительно компьютера?

Например: активный ввод, чтение, отсутствие, перерыв, возвращение.

Сонливость отвечает на вопрос:

Насколько человеку хочется спать и насколько трудно сохранять бодрствование?

Субъективное утомление отвечает:

Насколько сам человек ощущает снижение сил и способности эффективно продолжать работу?

Наблюдаемая рабочая нагрузка — внутренний интегральный вывод нашей программы по доступным каналам.

Нельзя писать:

PASSIVE_WORK = усталость
ACTIVE_WORK = бодрость

Это принципиально неверно. Уставший человек способен продолжать быстро печатать; бодрый сотрудник может двадцать минут спокойно читать PDF и не двигать мышью.

Чего продукт не должен делать

Он не должен:

- определять личность по лицу;
- оценивать лояльность, мотивацию или «желание работать»;
- выставлять сотруднику рейтинг продуктивности;
- отправлять руководителю индивидуальный «процент усталости»;
- хранить текст набранных сообщений;
- постоянно записывать видео;
- определять эмоции вроде «грустит», «злится», «плачет» по выражению лица;
- выдавать медицинские диагнозы;
- утверждать, что человек «устал на 72%».

Последнее особенно важно. Научная литература поддерживает ряд **связанных с утомлением маркеров**, но это не превращает их в универсальный метр физиологической усталости. Например, PERCLOS является одним из наиболее валидированных визуальных индикаторов сонливости, но даже он имеет ограничения и зависит от определения закрытия глаза, условий измерения и степени сонливости. ³

Кроме того, использование биометрии для вывода эмоций работников создаёт серьёзные юридические ограничения. В действующей редакции EU AI Act использование ИИ для вывода эмоций человека на рабочем месте запрещено, кроме узких случаев, связанных с медицинскими или safety-целями. Поэтому предложение «камера увидела, что сотрудник плачет, и решила, что ему эмоционально плохо» сознательно исключено из архитектуры. ⁴

Если в будущем нам нужен эмоциональный контекст, человек должен **сам добровольно сообщить** его, например:

Что сейчас больше всего мешает?

Ничего
Сонливость

Усталость глаз
Трудно сосредоточиться
Напряжение / стресс
Эмоционально тяжело

«Следить за всеми движениями» не означает тотальную запись

В первоначальных требованиях присутствовала идея отслеживать как можно больше поведения «и в реальной жизни, и на компьютере». В производственной архитектуре это надо трактовать как **мультимодальное измерение ограниченного набора заранее определённых признаков**, а не тотальное наблюдение.

Например, MediaPipe технически позволяет получать сотни геометрических landmarks лица и тела в реальном времени. Современный Face Landmarker выдаёт сотни трёхмерных ориентиров лица, а Holistic Landmarker объединяет лицо, тело и кисти. Но наличие такой технической возможности не означает, что продукт должен хранить или анализировать всё подряд. ⁵

Правильный принцип:

собирать минимальный набор признаков, для каждого из которых заранее определена конкретная роль в алгоритме.

История разработки: от первых датчиков до Step 7

Разработка шла не линейно. Многие важные архитектурные решения появились именно после того, как ранние прототипы показали свои недостатки.

Начальная техническая проверка

Первым делом была проверена среда Python.

На рабочей машине были подтверждены:

```
OpenCV: 4.13.0  
MediaPipe: 0.10.14
```

Текущий `requirements.txt` проекта фиксирует:

```
Python: рекомендуется 3.11  
mediapipe==0.10.14  
opencv-python>=4.8,<5  
numpy>=1.24
```

Датчик присутствия лица

Был создан `app_core/face_presence.py`.

Он открывал веб-камеру через OpenCV, считывал кадр и рассчитывал:

```
face_detected
data_quality
detection_confidence
brightness
sharpness
face_box
reason
```

Для диагностики появился `demo_face_presence.py`.

Первый рабочий интерфейс показывал непосредственно поверх видео:

```
FACE DETECTED
Quality: ...
Confidence: ...
Brightness: ...
```

Именно здесь было принято первое важное UX-решение: **confidence, quality, brightness и sharpness — показатели для инженера, а не для рядового сотрудника.**

`Confidence` показывает уверенность детектора в найденном лице, но не является вероятностью усталости. `Quality` — наша инженерная оценка пригодности кадра. `Brightness` — средняя яркость изображения. Пользователю эти числа почти ничего не сообщают.

Поэтому было решено:

технические числа остаются в диагностике, а основной интерфейс показывает только понятный результат.

Общий монитор сессии

Следом появился `app_core/session_monitor.py`, который объединил:

- камеру;
- Windows idle time;
- обезличенный ритм клавиатуры;
- state machine.

Внутренние состояния:

```
UNKNOWN
ACTIVE_WORK
PASSIVE_WORK
AWAY
```

```
BREAK
RETURNING
```

Появился `demo_session_monitor.py`.

Состояние `UNKNOWN` первоначально отображалось как:

«Недостаточно данных»

После пользовательского теста формулировка была признана слишком технической. Для пользовательского интерфейса её заменили на:

«Определяю состояние...»

с конкретной дополнительной причиной, например:

«Лицо не видно».

Неудачная клавиша `P`

В раннем демо ручной перерыв включался клавишей `P`.

Проблема обнаружилась сразу: чтобы нажать `P`, сотрудник должен был перейти в окно программы. Для фонового рабочего приложения это неприемлемо.

Поэтому:

- клавиша `P` была убрана из пользовательской концепции;
- глобальный хоткей тоже не считается хорошим основным решением;
- ручной перерыв в производственной версии должен включаться через обычный интерфейс или системный tray;
- автоматическое отсутствие пользователя должно определяться независимо.

Различение чтения и отсутствия

Возникла следующая проблема:

Человек может долго читать и совершенно не двигать мышью.

Нельзя считать это бездействием.

Поэтому был создан `app_core/visual_attention.py`, использующий Face Mesh, landmarks глаз, iris landmarks и геометрию головы.

Он получает:

```
gaze_on_screen
horizontal_gaze_ratio
head_turn_ratio
```

```
head_vertical_ratio
left_eye_opening_ratio
right_eye_opening_ratio
left_eye_quality
right_eye_quality
head_pose_quality
```

Направление взгляда было введено **не как прямой маркер усталости**, а прежде всего как контекст:

```
лицо видно
+ нет ввода
+ человек ориентирован на экран
→ PASSIVE_WORK
```

Разделение рабочего контекста и утомления

Затем была выявлена фундаментальная ошибка самой концепции:

`PASSIVE_WORK` вовсе не означает утомление.

После этого архитектура была разделена на два слоя.

Контекст работы:

```
ACTIVE_WORK
PASSIVE_WORK
AWAY
BREAK
RETURNING
UNKNOWN
```

Оценка нагрузки/утомления:

отдельный механизм, использующий совокупность признаков.

Это решение действует до сих пор и не должно быть отменено.

Обезличенная клавиатура

Был разработан `typing_activity.py`.

Программа не сохраняет текст и названия клавиш. Она превращает клавиатуру в временной сигнал:

```
keys_per_minute
mean_interval_sec
```

```
median_interval_sec
rhythm_cv
correction_ratio
long_pause_count
burst_count
```

Исследования реальной офисной работы показывают, что динамика печати действительно может меняться с умственным утомлением: в одном полевом исследовании сначала росло количество исправлений при сохранении скорости, а позже снижались и скорость, и точность. Однако эффекты зависят от человека, времени суток, дня недели и характера задачи, поэтому клавиатура не должна использоваться как единственный «детектор усталости». ⁶

Из этого выросла идея **персонального baseline**.

Персональная норма

Были созданы:

```
app_core/baseline_engine.py
app_core/typing_baseline.py
```

Основная продуктовая установка:

сравнивать человека прежде всего с ним самим, а не со среднестатистическим сотрудником.

Согласно исходному требованию, первичная калибровка рассчитана на:

5 рабочих дней
первые 120 минут рабочего дня

В текущем коде `PersonalBaselineEngine` действительно содержит:

```
calibration_days = 5
baseline_window_minutes = 120
min_windows_per_feature = 30
min_samples_for_stats = 5
max_windows_per_feature = 500
```

Признаки описываются с помощью медианы, MAD и robust z-score.

При этом сами **5 дней и 2 часа являются продуктовой гипотезой, которую ещё предстоит проверить**, а не научно доказанным универсальным сроком.

Развитие глазного канала

Следующим крупным направлением стал анализ глаз.

Первый вариант был слишком медленным: анализ выполнялся примерно раз в `0,35 с`, то есть около 2,9 измерения в секунду.

Для обычного короткого моргания это неприемлемо — значительная часть события способна полностью пройти между двумя анализами.

После этого компьютерное зрение было вынесено в отдельный высокочастотный поток.

В текущем UI:

```
целевой visual FPS: 20
основная оценка UI: 1 раз/с
preview: до 10 раз/с
```

То есть частота GUI теперь не определяет частоту анализа глаз.

Проблема с неполными морганиями

После повышения FPS появилась другая проблема: обычные моргания иногда классифицировались как `INCOMPLETE_BLINK`.

Поначалу возник соблазн вручную менять пороги:

```
0.88
0.90
0.92
...
```

Но было принято правильное решение остановить ручную подгонку без ground truth.

Появилась отдельная система валидации.

Ocular validation

Создан инструмент:

```
ocular_validation.py
```

с командами:

```
capture
review
report
quality
```

```
delete-video  
list
```

Validation mode — **единственный режим**, где допускается временная запись видео.

Он сохраняет:

```
validation video  
signals.csv  
metadata  
quality_report  
ручную разметку  
найденные события
```

и рассчитывает:

```
precision  
recall  
F1  
false positives  
false negatives  
FPS  
signal coverage
```

После завершения разметки видео можно удалить, сохранив только числовые признаки и результаты.

Ошибка quality gate и персональная поза

В одной реальной validation-сессии выяснилось, что около 81% кадров отбрасываются по геометрии головы, а полезное покрытие составляет лишь около 18,1%.

Камера при этом фактически работала нормально.

Причина: старая формула сравнивала всех людей с условной универсальной вертикальной геометрией лица.

Её заменили персональной калибровкой:

- первые секунды пользователь спокойно смотрит на экран;
- рассчитываются индивидуальные median/MAD для `head_turn_ratio` и `head_vertical_ratio`;
- дальше оценивается отклонение от **собственной нейтральной позы**.

В одной из последующих пользовательских записей после этого было получено:

```
QUALITY: PASS
coverage: 98.6%
569/577 пригодных кадров
```

и только восемь отклонений по ракурсу.

Это важный результат, но его нельзя неправильно интерпретировать:

98,6% coverage доказывает только то, что глазной сигнал почти всё время был доступен. Это не 98,6% точности распознавания морганий.

До завершения ручной разметки и оценки precision/recall/F1 Vision остаётся диагностическим.

Внутренняя инженерная цель, которую мы обсуждали, — приблизиться к precision/recall порядка 90%+ в заявленных условиях и иметь post-calibration coverage не ниже 85%. Это наша цель валидации, а не уже достигнутая характеристика продукта.

Первый сквозной MVP

После долгой разработки сенсоров был сделан важный продуктовый шаг: перестать бесконечно строить измерители и собрать работающую цепочку:

```
наблюдение
→ решение
→ рекомендация
→ действие пользователя
→ обратная связь
```

Появились:

```
assistant_app.py
app_core/recovery_engine.py
app_core/recovery_storage.py
```

Был создан `--demo` с ускоренными временными порогами.

Проблема demo mode

В демо система могла через несколько десятков секунд показать рекомендацию, что выглядело как «программа обнаружила усталость».

На самом деле срабатывал только ускоренный таймер.

Поэтому демо было явно помечено:

это проверка интерфейса и механизма уведомлений, а не вывод об усталости.

Эволюция интерфейса

Сначала главный экран напоминал панель разработчика.

Затем были убраны:

- постоянные технические таблицы;
- большая панель показателей;
- постоянно видимое видео.

Было принято главное UX-правило:

сотруднику нужны не 50 параметров, а три ответа.

1. СОСТОЯНИЕ
2. ОСНОВАНИЕ
3. РЕКОМЕНДАЦИЯ

Все технические показатели отправлены в:

Подробнее ▾

Step 7

На текущем этапе сформирована сборка Step 7.

В ней:

- пять пользовательских уровней;
- отдельная надёжность оценки;
- KSS;
- Samn-Perelli;
- два ползунка;
- подробные понятные описания каждого значения;
- сравнение с соседними уровнями;
- скрытая камера;
- отдельное диагностическое preview;
- вкладки Сессия / Печать / Глаза / Решение ;
- гистерезис итогового состояния;
- глазной канал заблокирован от принятия решений.

На текущем архиве повторно запущен тестовый набор:

Ran 113 tests
OK

Самое последнее замечание

В последнем пользовательском тесте выяснилось, что мы слишком глубоко спрятали время рабочей сессии.

Backend уже имеет:

```
continuous_work_sec
```

но на основном экране оно явно не отображается.

Следующая UI-правка зафиксирована:

Непрерывная работа: 01:59:47
Оценка обновлена: сейчас

Это не новый четвёртый информационный блок, а компактный контекст под текущим состоянием.

Научная модель: какие признаки должны участвовать и как их понимать

Главная научная предпосылка проекта — **утомление не имеет одного универсального наблюдаемого симптома**.

NIOSH подчёркивает, что утомление связано не только с сонливостью: на него могут влиять рабочий график, длительность бодрствования, стресс, умственная и физическая нагрузка, монотонность и продолжительность задачи. Поэтому технология обнаружения fatigue должна использоваться в контексте заранее определённой задачи и, при необходимости, нескольких независимых измерительных каналов. ⁷

Для проекта принята следующая иерархия.

Канал	Что измеряет	Роль	Текущее состояние
Рабочая сессия	длительность непрерывной работы	контекст риска	работает
Windows input	время с последнего ввода	рабочий контекст	работает
Клавиатура	скорость, ритм, исправления, паузы	поведенческий канал	работает
Самооценка SP	субъективное утомление	сильный независимый канал	работает
Самооценка KSS	сонливость	сильный независимый канал	работает

Канал	Что измеряет	Роль	Текущее состояние
Лицо	присутствие	контекст	работает
Взгляд	ориентированность на экран	контекст чтения	работает
Открытие глаз	динамика век	будущий физиологический канал	diagnostic
PERCLOS	доля времени существенного закрытия	сонливость	diagnostic
Длительность закрытий	длинные закрытия глаз	сонливость	diagnostic
Blink duration	длительность морганий	дополнительный ocular marker	diagnostic
Blink frequency	частота морганий	слабее и зависит от задачи	diagnostic
Mouse dynamics	динамика мыши	будущий поведенческий канал	ещё нет
Поза/голова	статичность, наклоны, кивки	дополнительный контекст	частично
Время суток	циркадный контекст	будущий contextual channel	ещё не используется
Сон перед работой	sleep debt	будущий contextual channel	ещё не используется
Liveness	живой ли источник изображения	integrity/quality	ещё нет
Anti-automation	реальны ли события ввода	integrity/quality	частично

Почему глаза важны, но не являются всей системой

PERCLOS — доля времени, когда глаза значительно закрыты, — является одним из наиболее исследованных пассивных признаков сонливости. Продолжительные эпизоды закрытия век также учащаются и удлиняются при выраженном недосыпании. Исследования показывают, что длительность закрытий может быть особенно информативной. ⁸

При этом частоту морганий нельзя интерпретировать примитивно. Само использование визуального дисплея способно снижать частоту морганий и увеличивать долю неполных морганий; значение зависит от визуальной задачи. Поэтому схема:

стал моргать реже
→ устал

недопустима. 9

В будущем ocular channel должен использовать совокупность:

```
PERCLOS
blink duration
long closures
severe closures
eye opening baseline
incomplete blink statistics
signal coverage
left/right eye agreement
head pose quality
```

а не один счётчик морганий.

Почему клавиатура является дополнительным, а не главным каналом

Печать интересна тем, что она:

- уже происходит естественным образом;
- не требует от человека дополнительного датчика;
- хорошо персонализируется;
- позволяет получать длительную временную историю.

Но изменение печати может быть вызвано:

- сложной задачей;
- другим языком;
- необходимостью подумать;
- звонком;
- новым интерфейсом;
- спешкой;
- неудобной клавиатурой;
- характером текста.

Поэтому поведенческий сигнал должен считаться значимым только после сравнения с личной нормой, учёта контекста и подтверждения во времени. Исследование реальной офисной печати действительно обнаруживало изменения интервалов и исправлений с time-on-task, временем суток и днём недели, что одновременно поддерживает полезность канала и показывает его контекстную зависимость. 10

Продолжительность сессии — фактор риска, а не доказательство

Чем дольше продолжается задача, особенно монотонная, тем выше вероятность утомления, а перерывы или смена деятельности являются одним из способов уменьшить time-on-task fatigue. Но 70 минут работы сами по себе не доказывают, что конкретный человек уже устал. 11

Поэтому наша длительность сессии должна быть:

контекстом и одним evidence source, а не таймером, замаскированным под ИИ.

Это особенно важно, потому что текущие реальные пороги движения:

45 мин
70 мин
105 мин

пока являются **эвристиками MVP**. Они не должны преподноситься банку как научно найденные границы человеческого утомления.

NIOSH приводит более общие рекомендации о регулярных перерывах при требовательной работе, например каждые 1–2 часа, но также подчёркивает зависимость эффекта перерыва от длительности бодрствования, sleep debt, времени суток и time-on-task. ¹²

Самооценка: два независимых измерения

Мы решили не создавать собственную шкалу «усталость 0–10».

Вместо неё используются:

KSS 1–9 — сонливость.

Karolinska Sleepiness Scale была валидирована относительно EEG и поведенческих показателей сонливости. ¹³

Samn-Perelli 1–7 — субъективная усталость.

Обе шкалы одновременно использовались, например, в крупном полевом исследовании FAA 2026 года вместе с Psychomotor Vigilance Task. ¹⁴

При этом мы сделали UX-адаптацию, потому что сухие значения вроде **6** и **7** плохо различимы для обычного пользователя.

Зафиксированные формулировки KSS:

Балл	Короткое название	Подсказка в текущем прототипе
1	Максимально бодр	Я чувствую себя особенно бодрой, быстрее и живее обычного.
2	Очень бодр	Энергии много, внимание включается сразу, но это не мой абсолютный пик.
3	Нормально бодр	Обычное рабочее состояние. Спать совершенно не хочется.
4	Скорее бодр	Я всё ещё явно бодр, но уже нет ощущения высокой энергии.
5	Нейтрально	Я не чувствую ни бодрости, ни сонливости. Просто обычное нейтральное состояние.

Балл	Короткое название	Подсказка в текущем прототипе
6	Слегка сонно	Я уже замечаю сонливость, но желания лечь спать у меня ещё нет.
7	Хочется спать	Я определённо хочу спать, но мне пока не приходится заставлять себя держать глаза открытыми или бороться со сном.
8	Трудно сохранять бодрость	Мне уже приходится прикладывать усилие, чтобы не начать отключаться и продолжать нормально работать.
9	Борюсь со сном	Я реально могу начать засыпать: тяжело держать глаза открытыми и сохранять бодрствование.

Смысловые границы соответствуют структуре KSS: 6 — некоторые признаки сонливости; 7 — сонливость без усилий для сохранения бодрствования; 8 — уже требуется усилие; 9 — выраженная сонливость с борьбой со сном. ¹⁵

Зафиксированные формулировки Samn-Perelli:

Балл	Короткое название	Подсказка в текущем прототипе
1	Полностью свежа	Чувствую запас сил и могу работать на своём максимуме.
2	Много энергии	Работать легко, энергии много, но я не на абсолютном пике.
3	Нормальное состояние	Работа идёт обычно. Усталость практически не мешает.
4	Начинаю уставать	Энергии заметно меньше, но я всё ещё могу работать в обычном темпе без дополнительного усилия.
5	Усталость мешает	Чтобы сохранять обычный темп и концентрацию, мне уже приходится сознательно напрягаться.
6	Очень устала	Концентрация регулярно срывается, продолжать работу тяжело даже если стараюсь.
7	Истощена	Я сейчас не могу нормально и эффективно продолжать работу.

Исходная Samn-Perelli также строится как переход от полностью бодрого состояния через нарастающую усталость к состоянию, когда человеку уже трудно концентрироваться и эффективно функционировать. ¹⁶

Важно для следующего программиста: пользователь уже одобрил именно смысловое различие этих соседних уровней. Не заменять их обратно на расплывчатые:

немного устал
 средне устал
 сильно устал

без отдельного UX-обсуждения.

При этом нынешние фразы написаны в женском роде, поскольку тестировались с конкретным пользователем. Для массового банковского продукта потребуется либо грамматически нейтральная русская локализация, либо персонализация форм. Семантику границ менять нельзя без новой UX-проверки.

В будущем предусмотрена функция:

«Не могу выбрать между двумя»

Она задаёт один разделяющий вопрос.

Например, между KSS 7 и 8:

«Вам уже приходится прикладывать усилие, чтобы оставаться бодрым?»

Самооценка должна оставаться независимой

Программа не должна подсказывать:

«Выберите 7, потому что камера увидела закрытие глаз».

Иначе человек начинает сообщать системе то, что она уже сама ему сказала, и мы получаем искусственное согласование каналов.

Правильная схема:

Человек сообщает ощущение

↓

независимый канал

Программа измеряет глаза/печать/сессию

↓

независимые каналы

↓

сопоставление evidence

Пять пользовательских уровней

Мы сознательно отказались от искусственного процента.

В Step 7 определены пять **функционально различных** состояний:

Уровень	Название
1	Рабочее состояние устойчиво

Уровень	Название
2	Появляются признаки нагрузки
3	Нагрузка становится устойчивой
4	Нагрузка выражена
5	Восстановление приоритетно

Пять было выбрано не по дизайнерскому вкусу.

Каждому уровню соответствует отдельная стратегия:

- 1 → продолжать
- 2 → наблюдать
- 3 → профилактическая разгрузка становится уместной
- 4 → восстановительный перерыв обоснован
- 5 → восстановление должно иметь приоритет

Если в ходе будущей валидации выяснится, что нужно шесть функционально различимых действий, число можно изменить. Но добавлять уровни только ради ощущения точности нельзя.

Надёжность — отдельная ось

Недостаточно данных не является шестой степенью утомления.

Отдельно существуют:

```
AssessmentReliability.LIMITED
AssessmentReliability.MODERATE
AssessmentReliability.HIGH
```

Это отвечает на другой вопрос:

насколько качественными и независимыми являются данные, на которых построено текущее решение?

Итог строится по evidence, а не по одной сумме

Текущая концепция:

```
каждый канал
→ evidence
→ severity
→ persistence
→ data quality
→ decision_ready
```

→ согласование источников
→ workload level

Например:

80 минут работы

сами по себе могут быть ранним/умеренным признаком.

Но:

80 минут работы
+ два устойчивых изменения печати
+ SP = 5

— гораздо более сильное сочетание.

А:

KSS = 8
+ устойчиво выросла длительность закрытия глаз
+ хороший ocular coverage

даёт независимое согласование субъективного и физиологического канала.

Гистерезис

Результат нельзя обновлять буквально вслед за каждым шумным измерением.

Плохой интерфейс:

12:01 уровень 2
12:02 уровень 3
12:03 уровень 2
12:04 уровень 3

Поэтому Step 7 имеет гистерезис.

В реальном режиме:

level_rise_confirmations = 3
level_drop_confirmations = 10

Экстремальный сигнал может повысить состояние немедленно.

Это означает, что при секундном цикле:

- обычный новый сигнал меняет верхний уровень примерно за несколько секунд;
- понижение обычно требует порядка 10 секунд устойчивого улучшения;
- крайняя самооценка может повлиять на ближайшем цикле.

Как быстро обновляются разные каналы

Это принципиально разные временные масштабы:

Источник	Текущая скорость
Основной <code>assessment</code>	1 Гц
Camera preview	до 10 Гц
Vision worker	целевые 20 FPS в UI
Windows idle	фактически доступен по запросу
Самооценка	после <code>Сохранить</code> , следующий <code>assessment</code> $\leq$ примерно 1 с
Окно печати	60 с
Минимум печати	~50 с + 25 релевантных событий
Baseline sample	не чаще 1/мин
Initial baseline	до 5 рабочих дней
Ocular event detection	кадровый поток
Blink/PERCLOS statistics	временные окна
Workload hysteresis	несколько последовательных <code>assessment</code> -циклов

Последняя самооценка сейчас считается актуальной **два часа**:

```
now - self._self_report_at <= 2 * 60 * 60
```

Мы уже признали это слишком грубой эвристикой.

Следующая версия должна разделять:

```
значение  
время ввода  
возраст данных  
вес/актуальность
```

и показывать пользователю, например:

Самооценка: 14 минут назад

Конкретную функцию decaу пока нельзя выбирать из головы — её нужно проверить экспериментально.

Текущая backend-архитектура и поток данных

Текущий архив имеет следующую основную структуру:

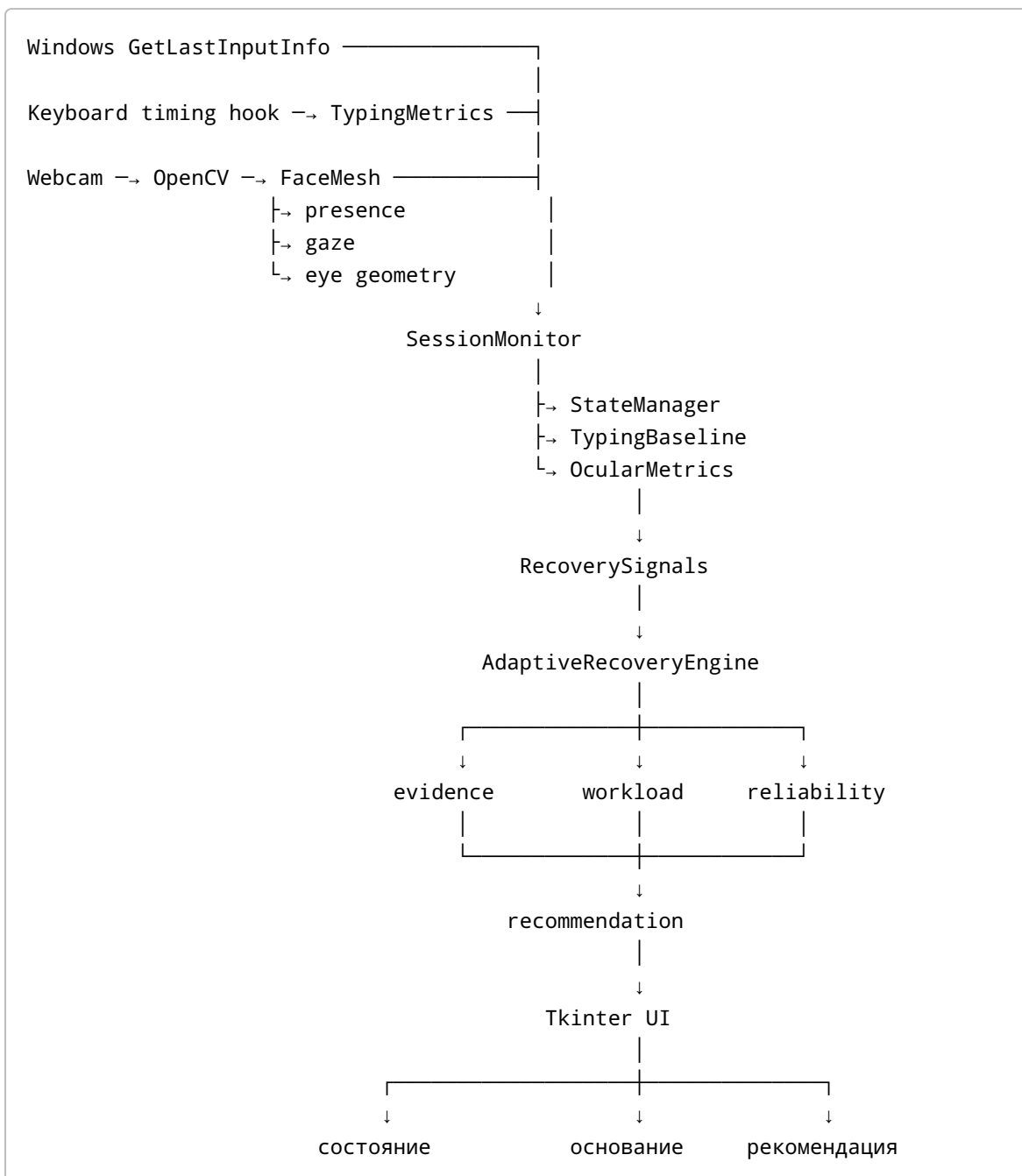
```
assistant_app.py
main.py
ocular_validation.py

app_core/
  baseline_engine.py
  face_presence.py
  input_activity.py
  ocular_activity.py
  ocular_validation.py
  recovery_engine.py
  recovery_storage.py
  session_monitor.py
  state_machine.py
  storage.py
  typing_activity.py
  typing_baseline.py
  visual_attention.py

tests/
  test_assistant_ui.py
  test_baseline_engine.py
  test_input_activity.py
  test_ocular_activity.py
  test_ocular_validation.py
  test_ocular_validation_ui.py
  test_recovery_engine.py
  test_recovery_storage.py
  test_session_monitor.py
  test_state_machine.py
  test_typing_activity.py
  test_typing_baseline.py
  test_visual_attention.py
```

Общий data flow

Сейчас архитектуру можно представить так:



`input_activity.py`

Использует Windows `GetLastInputInfo`.

API Microsoft возвращает время последнего пользовательского input event и предназначен, в частности, для определения idle time. Он является session-specific, что соответствует нашей desktop-сценарной модели. ¹⁷

Мы не считываем текст через этот API.

Получается примерно:

```
input_idle_sec = сколько секунд не было ввода
```

`typing_activity.py`

Использует низкоуровневый keyboard hook, но не превращает его в keylogger.

Система классифицирует события в нужные статистические категории и сохраняет интервалы, а не содержимое.

Особое внимание уделено инъецированным событиям. Windows-флаг `LLKHF_INJECTED` позволяет определить, что клавиатурное событие было создано программным вводом; текущий код такие события отбрасывает. ¹⁸

Это уже частичная anti-automation-защита, но **не полноценный anti-bot**.

`face_presence.py`

OpenCV `VideoCapture` получает кадры веб-камеры; OpenCV официально поддерживает захват последовательных кадров с камеры через `VideoCapture`. ¹⁹

Кадр зеркалируется и в памяти рассчитываются:

```
brightness  
sharpness  
frame quality
```

Текущая простая формула качества учитывает освещение и Laplacian-based sharpness.

Эти показатели — только quality gate.

`visual_attention.py`

MediaPipe Face Mesh получает landmarks.

Используются, среди прочего:

```
LEFT_EYE_CORNERS  
RIGHT_EYE_CORNERS  
vertical eye pairs  
LEFT_IRIS  
RIGHT_IRIS  
cheeks  
nose  
chin
```

Из них рассчитываются:

```
eye opening
horizontal gaze
head turn
vertical head geometry
eye quality
head pose quality
face box
```

Современная официальная MediaPipe Face Landmarker также построена на плотной карте лица и возвращает сотни трёхмерных landmarks, то есть выбранное направление технически соответствует назначению библиотеки для continuous visual streams. ²⁰

Важно: **не использовать blendshapes для рабочего emotion recognition.**

`ocular_activity.py`

Получает нормализованное открытие каждого глаза отдельно.

Поддерживаются или проектируются классы:

```
BILATERAL_BLINK
INCOMPLETE_BLINK
LEFT_WINK
RIGHT_WINK
LONG_BILATERAL_CLOSE
UNCERTAIN
```

Текущие диагностические параметры включают:

```
window_sec = 60
calibration_min_samples = 60
calibration_min_sec = 3
min_observation_sec = 10
min_eye_quality = 0.55
min_head_pose_quality = 0.35

closed_ratio_threshold = 0.55
full_blink_ratio_threshold = 0.82
partial_ratio_threshold = 0.90
open_ratio_threshold = 0.94

min_blink_sec = 0.03
max_blink_sec = 0.70
long_closure_sec = 1.0
severe_closure_sec = 2.0
bilateral_sync_tolerance_sec = 0.12
```

```
min_reliable_fps = 15
min_signal_coverage = 0.70
```

Эти значения не являются производственными медицинскими границами. В коде они прямо помечены как временные диагностические.

state_machine.py

Конфигурация:

```
active_input_threshold_sec = 8
away_input_threshold_sec = 20
away_face_threshold_sec = 12
returning_duration_sec = 60
low_quality_threshold = 0.45
transition_debounce_sec = 2
```

Логика построена на принципах:

```
движение мыши недавно
→ active work

лицо видно + взгляд к экрану
→ passive work допустим

мышь давно не двигалась
≠ автоматически away

лицо давно отсутствует
+ давно нет input
→ away
```

screen_locked уже существует в структуре PresenceSignals, но в текущем SessionMonitor передаётся:

```
screen_locked=False
```

То есть реальное обнаружение блокировки Windows **ещё не реализовано**.

session_monitor.py

Это центральный sensor fusion слой.

В Step 7 он:

- открывает камеру;
- запускает клавиатурный sensor;

- запускает отдельный visual thread;
- собирает Windows idle;
- получает глазные показатели;
- обновляет state machine;
- пытается обновлять typing baseline;
- выдаёт `SessionSnapshot` .

В UI используется:

```
target_visual_fps=20.0
include_diagnostic_frame=False
```

Сам визуальный поток хранит только последний кадр в оперативной памяти для анализа/preview.

Есть защита от устаревшего изображения:

```
visual_age > 1 sec
→ face/gaze считаются unavailable
```

Важный текущий недостаток: камера технически обязательна

Концептуально мы хотим в будущем иметь:

```
Basic mode
→ работает без Vision

Vision mode
→ дополнительный ocular channel
```

Но нынешний `SessionMonitor.start()` непосредственно вызывает:

```
self.face_sensor.start()
```

и ошибка открытия камеры способна остановить старт.

Следовательно, **graceful degradation без камеры пока не реализован**.

Это одна из задач будущего hardening.

`typing_baseline.py`

В baseline принимается только `ACTIVE_WORK` .

Это правильное решение: если человек читает и не печатает, нельзя записывать:

```
typing speed = 0
```

как будто это его новый нормальный темп.

Для baseline требуется:

```
min observation = 50 sec  
target relevant keys = 25  
sample interval = 60 sec
```

`baseline_engine.py`

Предназначен не только для печати.

Внутренне уже предусмотрены каналы:

```
TYPING  
MOUSE  
OCULAR  
POSTURE  
GENERIC
```

и контексты:

```
TYPING  
READING  
MIXED
```

Для визуальных baseline-окон предусмотрено поле:

```
liveness_score
```

и минимальный порог:

```
0.80
```

но реальный liveness sensor пока не подключён.

Технический долг baseline

В старом `BaselineObservation` до сих пор существует:

```
self_report_fatigue_0_10
```

при том что Step 7 уже перешёл на:

```
Samn-Perelli 1-7  
KSS 1-9
```

Это legacy-поле нужно мигрировать.

Кроме того, текущий typing baseline не полностью синхронизирован с новой Step 7 самооценкой, поэтому фильтрация baseline по текущему SP требует доработки.

```
recovery_engine.py
```

Это текущий decision engine.

Типы рекомендаций:

```
NONE  
EYE_REST  
MICROBREAK  
RECOVERY_BREAK
```

Источники:

```
SESSION  
TYPING  
OCULAR  
SELF_REPORT
```

Пять уровней `WorkloadLevel` описаны выше.

В реальном режиме текущие параметры:

```
eye_rest_after_sec          = 45 min  
microbreak_after_sec        = 70 min  
recovery_break_after_sec    = 105 min  
meaningful_break_sec        = 180 sec  
alert_cooldown_sec          = 25 min  
  
typing_evaluation_interval   = 10 sec  
typing_persistence_samples   = 3  
typing_required_positive     = 2  
typing_z_threshold           = 1.5
```

```
ocular_decisions_enabled      = False  
  
level_rise_confirmations     = 3  
level_drop_confirmations     = 10
```

B demo:

```
45 sec  
70 sec  
105 sec  
15 sec meaningful break  
30 sec cooldown
```

Эти временные значения намеренно ускорены и не имеют физиологического значения.

Текущие правила typing evidence

После готового baseline:

```
KPM robust z <= -1.5  
→ slowdown  
  
rhythm CV robust z >= +1.5  
→ rhythm instability  
  
correction ratio >= +1.5  
→ corrections increased  
  
long pause rate >= +1.5  
→ pauses increased
```

Признак должен появиться минимум в двух из последних трёх evaluation samples, чтобы считаться устойчивым.

1.5 тоже пока инженерный порог, который нужно валидировать.

Текущая самооценка в engine

Samn-Perelli:

```
1-3 → severity 0  
4   → 1  
5   → 2  
6-7 → 3
```

KSS:

```
1-5 → 0
6 → 1
7 → 2
8-9 → 3
```

SP=7 или KSS=9 получает специальный очень сильный статус.

Это уже значительно лучше старого правила `>=7`, но дальнейшая калибровка по экспериментальным данным всё равно нужна.

Текущая логика уровня

Упрощённо:

```
very high self-report
или severe eye closure
→ RECOVERY_PRIORITY

strong self-report
или ≥2 независимых source с умеренным/сильным evidence
или very long session
→ EXPRESSED

long session
или ≥2 persistent typing anomalies
или moderate self-report + другой source
→ SUSTAINED

любое decision-ready evidence
→ EARLY

иначе
→ STABLE
```

Но `ocular_decisions_enabled=False`, поэтому глазной канал в текущем продукте фактически не должен поднимать уровень.

`recovery_storage.py` и `storage.py`

Используется локальная SQLite-база:

```
data/fatigue_assistant.sqlite3
```

SQLite подходит для текущего локального прототипа как лёгкое файловое хранилище без отдельного server process. ²¹

Хранятся:

- profiles;
- sessions;
- metric windows;
- aggregate metric values;
- baseline samples;
- state events;
- recommendations;
- self-reports;
- feedback.

Не должны храниться:

typed text
raw keyboard names
screenshots
ordinary video
ordinary camera frames

`recovery_self_reports` уже хранит SP/KSS, а legacy 0–10 пока сохраняется для обратной совместимости.

Пользовательский интерфейс, рекомендации и будущая роль ИИ

Интерфейс проектируется для **взрослого работающего человека, который может быть уже утомлён и не хочет читать длинные объяснения.**

Поэтому договорённость о стиле жёсткая:

формально, понятно, коротко, без детского тона, без юмора в основных статусах и без избитых медицинских формулировок.

Фразы вроде:

«Всё спокойно»

были признаны слишком неформальными.

Фраза:

«Усталость не обнаружена»

тоже нежелательна: она клиширована и создаёт ложное впечатление диагностической достоверности.

Основной экран: ровно три смысловых блока

Состояние

Например:

Рабочее состояние устойчиво

Основание

Не более 1–2 наиболее значимых причин:

Рабочая сессия: 38 минут.

Ритм работы соответствует личному диапазону.

Рекомендация

Можно продолжать работу в обычном режиме.

или:

Рекомендуется короткий восстановительный перерыв.

Дополнительно компактно:

Непрерывная работа: 01:17:43
Оценка обновлена: сейчас

Это последняя согласованная доработка и пока ещё не внесена в переданную Step 7 сборку.

Подробнее

Пользователь хочет иметь возможность раскрыть технический механизм.

В Step 7 вкладки:

Сессия
Печать
Глаза
Решение

Туда допускается выводить десятки параметров:

```
state
state_reason
seconds_in_state
continuous_work_sec
```

input_idle_sec

face_detected
gaze_on_screen
visual_age_sec
frame quality
brightness
sharpness

left eye opening
right eye opening
left/right quality
PERCLOS
blink rate
blink duration
long closure count
severe closure
FPS
coverage

keys/min
median interval
mean interval
rhythm CV
correction ratio
long pauses
bursts

baseline median
MAD
robust z
baseline readiness
baseline progress

evidence codes
severity
decision_ready
raw workload
stable workload
hysteresis counter
cooldown
snooze
self-report
self-report age
reliability

Здесь «50 показателей» — допустимо, поскольку это осознанно открываемая диагностика.

Последняя проблема с интерфейсом решения

На последнем скриншоте возникла потенциально серьёзная UX-несогласованность:

Состояние:

Нагрузка выражена

Основание:

Очень длинная непрерывная рабочая сессия

Рекомендация:

Продолжайте работу в обычном режиме

В коде понятно, почему такое возможно: если активного `RecoveryRecommendation` сейчас нет, `_render_action()` для всех уровней кроме `EARLY`, `AWAY` и `BREAK` способен показать:

«Продолжайте работу в обычном режиме».

Это неправильно семантически.

Если уровень уже `EXPRESSED`, отсутствие новой toast-рекомендации из-за cooldown не должно превращаться в противоположный совет.

Следующая версия должна разделить:

текущее рекомендуемое действие

и:

нужно ли сейчас создавать новое уведомление

Cooldown должен подавлять повторный toast, **но не менять смысл рекомендации на главном экране.**

InterventionEngine — следующий большой продуктовый модуль

Главная будущая идея — не ограничиваться:

«Сделайте перерыв».

Программа должна отвечать:

Какой перерыв сейчас наиболее уместен?

Это соответствует самому смыслу мультимодальной системы: если причины различаются, вмешательство тоже может различаться. NIOSH перечисляет среди fatigue-management interventions корректировку перерывов, schedule, nap strategies, обучение sleep hygiene и другие меры; time-on-task fatigue также может уменьшаться за счёт перерыва или смены задачи. ²²

Первая версия `InterventionEngine` может иметь, например:

```
VISUAL_DISTANCE_BREAK  
LIGHT_MOVEMENT_BREAK  
COGNITIVE_DISENGAGEMENT_BREAK  
HIGH_SLEEPINESS_RECOVERY  
ERGONOMIC_RESET
```

Но название техники — только условное. Для каждой должна существовать экспертная карточка:

```
technique_id  
target_load  
required_evidence  
contraindications  
minimum_duration  
maximum_duration  
instructions  
evidence_level  
source  
source_version  
validation_status
```

Перерывы должны быть evidence-based

Есть основания считать короткие перерывы полезными для субъективного восстановления: метаанализ 22 экспериментальных исследований обнаружил улучшение vigor и снижение fatigue после micro-breaks, но общего значимого улучшения performance не выявил; эффект на performance зависел от контекста и длительности перерыва. Следовательно, нельзя обещать, что любая 90-секундная пауза автоматически повысит производительность. ²³

OSHA также приводит рекомендации по регулярным коротким паузам при компьютерной работе и отмечает данные о снижении зрительного и мышечно-скелетного дискомфорта без снижения продуктивности. ²⁴

Поэтому нынешние:

```
90 секунд  
3 минуты  
5 минут
```

следует считать **prototype intervention durations**, а не окончательными научными рецептами.

Роль ИИ

Мы договорились, что LLM **не должна сама решать, насколько человек устал и какую медицинскую рекомендацию придумать.**

Правильная архитектура:

```
сенсоры
↓
детерминированный Evidence Engine
↓
Workload Level
↓
детерминированный InterventionEngine
↓
выбор допустимой экспертной техники
↓
ИИ
↓
понятная персонализированная формулировка
```

Например:

```
InterventionEngine:
VISUAL_DISTANCE_BREAK
duration = 90 sec

Основания:
длительная экранная работа
ocular strain pattern
хорошее качество данных
```

ИИ превращает это в:

«Основная нагрузка сейчас приходится на зрительный канал. На короткое время уберите экран из поля зрения и переведите взгляд на дальние объекты.»

Knowledge base для ИИ

Планируется загружать в систему только отобранные материалы:

- официальные occupational-health рекомендации;
- профессиональные guidelines;
- peer-reviewed исследования;
- внутренне одобренные экспертами документы.

LLM получает **retrieval context из этой базы**, а не свободный доступ к интернету.

Поскольку исходное требование продукта — strict on-device, приоритетная производственная архитектура — локальная модель/RAG. Использование внешнего API допустимо рассматривать только как совершенно отдельный режим после банковской ИБ-экспертизы и без передачи raw sensor data.

Будущий «человеческий» ассистент

Позже продукт может писать спокойные человеческие комментарии и позволять короткий диалог.

Но он не должен говорить:

«Я вижу, что вы очень расстроены».

Лучше:

«По вашим текущим данным восстановление сейчас приоритетнее продолжения работы в прежнем темпе.»

Если сам пользователь сообщил:

«Эмоционально тяжело»,

тогда ИИ может выбрать из проверенной базы безопасный мягкий сценарий.

Идея «если человек рыдает — посоветовать ему поплакать ещё» была переосмыслена: **ИИ не должен назначать психологические действия на основании визуального emotion inference.**

Персонализация вмешательств

После перерыва пользователь должен иметь возможность сообщить:

Помогло
Помогло частично
Не помогло
Неуместно

С накоплением истории можно оценивать:

состояние X
+ техника A
→ насколько часто помогала

состояние X
+ техника B
→ насколько часто помогала

В будущем это позволит выдавать:

«При похожем состоянии вам раньше чаще помогал короткий перерыв с движением.»

Это потенциально один из наиболее ценных компонентов продукта.

Конфиденциальность, ИБ, anti-spoofing и ограничения корпоративного мониторинга

Для банка эта часть не второстепенная. Она должна рассматриваться как часть самой архитектуры.

В июле 2026 года Банк России опубликовал актуальные методические рекомендации по информационной безопасности ИИ для финансового рынка. В них рассматриваются модель угроз, риски моделей и компонентов, минимизация персональных данных, обезличивание, ограничение потоков, гарантированное стирание, принцип минимальных привилегий и оценка open-source компонентов. Это очень близко к тому, что нужно закладывать в производственную версию нашего приложения. ²⁵

Принцип On-Device

В обычном режиме:

```
камера
→ RAM
→ landmarks/features
→ кадр более не нужен

keyboard
→ временные интервалы/категории
→ текст не формируется

decision
→ локально

database
→ локально
```

На центральный сервер **не должны** отправляться:

```
raw video
camera frames
screenshots
typed text
raw keyboard log
raw mouse path
raw face landmark stream
```

Validation mode — отдельное исключение

Для научной разметки морганий изображение действительно требуется.

Поэтому есть отдельный явно запущенный:

```
ocular_validation.py capture
```

Это не рабочий режим сотрудника.

Должно сохраняться принципиальное разделение:

PRODUCTION

→ no video recording

VALIDATION

→ explicit consent + local temporary recording

После разметки видео удаляется отдельной командой.

Локальная база пока не зашифрована как production vault

SQLite хранится локально, но само по себе это не означает защищённое хранение.

Для банковской версии понадобятся:

- encryption at rest;
- привязка к Windows user/profile;
- управление ключами через OS mechanisms;
- минимальные ACL;
- retention policy;
- функция удаления личной истории;
- audit trail конфигурации;
- versioned schema;
- signed application updates.

Биометрические данные

Российское определение биометрических персональных данных связано с физиологическими и биологическими характеристиками, используемыми для установления личности. Роскомнадзор прямо указывает на такую связь с идентификацией. ²⁶

Наш продукт **не должен устанавливать личность по лицу**, что принципиально отличает его от biometric authentication.

Тем не менее это не означает, что любые facial landmarks автоматически становятся «юридически безопасными». Конкретная квалификация персональных и производных данных должна быть проведена юристами работодателя до deployment.

Для финансовых организаций Банк России отдельно публикует требования и рекомендации по защите biometric information в authentication-сценариях, включая методы обнаружения presentation attacks. Хотя наш сценарий не является аутентификацией, это демонстрирует уровень требований к обработке чувствительных лицевых данных в финансовой инфраструктуре. ²⁷

Liveness / anti-spoofing

Исходное требование пользователя:

система должна отличать живого человека от фотографии или видео, поставленного перед веб-камерой.

Сейчас полноценного механизма нет.

BaselineEngine уже имеет концептуальное поле:

```
liveness_score
```

но оно не обеспечено реальным PAD.

Не следует реализовывать liveness как примитив:

```
человек моргнул  
→ живой
```

Видео реального человека тоже содержит моргания.

Правильное направление — Presentation Attack Detection.

NIST отдельно тестирует passive software-based face PAD и показал, что такие алгоритмы действительно можно оценивать на spoof-атаках, но их эффективность существенно варьируется. ISO/IEC 30107-3:2023 задаёт принципы тестирования и отчётности PAD. 28

Для нашей задачи перспективны комбинации:

```
temporal landmark consistency  
естественные микродвижения головы  
3D/parallax consistency  
оптический поток  
изменение локального освещения  
replay/display artifacts  
moire/refresh artifacts  
неестественно повторяющаяся мимика
```

При подозрении можно использовать редкий active challenge, но постоянные «поверните голову вправо» несовместимы с фоновым продуктом.

Anti-spoofing здесь нужен **не для проверки личности**, а для integrity:

можно ли доверять визуальному каналу как наблюдению за реальным человеком?

Anti-autoclicker и симуляция активности

Сейчас уже есть полезный первый барьер:

```
LLKHF_INJECTED  
→ injected keyboard event  
→ исключается
```

Windows официально предоставляет этот флаг. ¹⁸

Но программные и особенно аппаратные макросы не всегда будут так очевидны.

Будущий `InputIntegrityAnalyzer` должен анализировать:

```
очень точную периодичность  
аномально низкий jitter  
повторение идентичных интервалов  
спектральные пики активности  
циклические последовательности  
точно повторяющиеся mouse trajectories  
несогласованность input и присутствия  
несогласованность input и work context  
долгую идеально периодическую активность
```

Важно: такой механизм не должен обвинять сотрудника в мошенничестве.

Его выход:

```
input channel integrity = low
```

а не:

«Сотрудник использует автокликер».

Будущий mouse channel

Пока `GetLastInputInfo` сообщает только, когда был последний ввод.

Полноценные mouse dynamics ещё не реализованы.

В будущем на устройстве можно вычислять агрегаты:

```
mouse path speed  
acceleration  
jerk  
pause distribution
```

```
click intervals
movement entropy
micro-corrections
trajectory curvature
repeated trajectories
```

и сразу удалять raw path.

Но этот канал должен пройти собственную валидацию. Связи характеристик мыши с психофизиологическими состояниями существуют в исследованиях, но сигнал сильно зависит от задачи; значит, использовать его как независимый универсальный маркер утомления нельзя без проверки именно в нашем рабочем сценарии.

Поза

Длительное статическое положение само по себе способно способствовать мышечному утомлению; NIOSH рассматривает длительные статические и неудобные позы как факторы эргономической нагрузки. ²⁹

Это обосновывает будущий `posture channel`, но его задача будет скорее:

эргономическая/мышечно-скелетная нагрузка,

а не прямой вывод о когнитивной усталости.

Время суток и сон

Это существенный будущий пробел.

Утомление зависит от:

- времени бодрствования;
- sleep debt;
- circadian phase;
- времени внутри смены;
- предыдущего восстановления.

NIOSH прямо перечисляет эти факторы, а полевое исследование FAA 2026 года показывает совместное изменение PVT, KSS и Samn-Perelli в зависимости от времени смены и циркадных условий. ³⁰

Поэтому позже нужен `FatigueContext`:

```
local time
workday start
shift length
time awake – только если пользователь добровольно сообщил
sleep duration – optional
```

```
recent break history
time on task
```

При этом нельзя превращать приложение работодателя в сборщик медицинской истории.

Производственная прозрачность

Для сотрудника должно быть очевидно:

- какие сенсоры активны;
- какие данные сохраняются;
- какие не сохраняются;
- как остановить мониторинг;
- как удалить личную историю;
- какие каналы участвовали в конкретном выводе;
- когда оценка была обновлена;
- насколько она надёжна.

Это не косметика, а основа доверия.

План валидации и дальнейшей разработки

Текущая версия уже значительно сложнее исходного демо, но до банковского production deployment остаётся несколько принципиальных этапов.

Что уже фактически завершено

Компонент	Статус
Python/OpenCV/MediaPipe foundation	готов
Face presence	работает
Frame quality	работает как диагностическая эвристика
Windows idle	работает
Context state machine	работает
Active/passive/away/returning	работает
Keyboard timing	работает
Raw-text privacy	реализована
Typing metrics	работает
Personal typing baseline framework	работает
5-day/2-hour baseline gating	заложено
FaceMesh / eye geometry	работает
Left/right eye separation	работает

Компонент	Статус
High-frequency visual worker	работает
Ocular event tracker	diagnostic
Ocular validation tools	работают
Personal head-pose quality calibration	работает
SQLite storage	работает
Recovery engine	работает как MVP
Demo mode	работает
Toast notifications	работают
Three-block UI	работает
KSS slider	работает
Samn-Perelli slider	работает
Five workload levels	работают
Reliability	работает
Hysteresis	работает
Подробнее	работает
Separate camera preview	работает
Automated tests	113 tests OK

Что доказано лишь частично

Vision quality

Мы доказали, что после персональной pose calibration можно добиться высокого signal coverage на конкретной проверенной сессии.

Мы ещё **не доказали** event detection accuracy на достаточном количестве:

- людей;
- камер;
- очков;
- освещений;
- ракурсов;
- типов морганий.

Typing

Логика реализована и научно правдоподобна, но наши конкретные:

60-sec window
z=1.5
2 of 3 persistence

ещё не валидированы как оптимальные.

Five-level workload model

Он объясним и функционально мотивирован, но границы ещё не подкреплены собственным полевым исследованием.

5-day baseline

Это разумное проектное требование, но его достаточность нужно доказать.

Самые ближайшие изменения — следующая версия Step 7

Перед добавлением новых сенсоров нужно исправить то, что уже выявлено.

На основной экран:

Непрерывная работа: HH:MM:SS
Оценка обновлена: сейчас

В [Подробнее](#) → [Решение](#) :

Последний полный пересчёт
Возраст самооценки
Возраст Vision
Статус typing window
Возраст work context
Последнее обновление baseline

Исправить конфликт:

Нагрузка выражена
+
Продолжайте работу

Cooldown notification и recommendation state должны быть разведены.

Переработать self-report TTL:

убрать грубые два часа как окончательное правило и ввести explicit freshness/decay.

Исправить legacy baseline:

```
fatigue_0_10  
→ Samn-Perelli 1-7
```

Затем — операционная надёжность

Нужно реализовать:

```
Windows screen lock detection  
camera unavailable fallback  
Basic mode without Vision  
system tray  
autostart  
graceful crash recovery  
low CPU mode  
adaptive visual FPS  
pause Vision while screen locked/away
```

Производственная логика не должна падать только потому, что камера занята Teams.

Затем — завершение ocular validation

Нужны размеченные реальные сессии.

Для каждого участника:

```
neutral calibration  
20+ normal blinks  
incomplete blinks where naturally available  
intentional winks for negative control  
head turns  
looking down/up  
glasses  
different lighting  
different camera distances
```

Ground truth размечается вручную.

Считать:

```
event-level precision  
event-level recall  
F1  
false positives/min  
false negatives
```

```
classification confusion
FPS
coverage
quality-gate rejection causes
```

Тренировочные и тестовые люди должны быть разделены: нельзя подобрать пороги на одном человеке и объявить точность на нём же.

Только после этого:

```
ocular_decisions_enabled = True
```

может рассматриваться как кандидат.

Затем — liveness и input integrity

Создать отдельные:

```
app_core/liveness.py
app_core/input_integrity.py
```

Liveness должен валидироваться по стандартному PAD-подходу, а не демонстрационным «моргнул — живой». ISO/IEC 30107-3:2023 является актуальной базой для тестирования/отчётности PAD. ³¹

Затем — мышь

Добавлять mouse dynamics **только как экспериментальный канал**.

Нужен тот же pipeline:

```
raw local events
→ aggregate features
→ raw path discarded
→ personal baseline
→ validation
→ only then decision-ready
```

Затем — InterventionEngine v1

Начать не с 50 советов, а примерно с 4–5 хорошо изученных техник.

Каждая техника проходит expert review.

На первом этапе AI не нужен для выбора — только deterministic rules.

Затем — экспертная RAG-база и локальный ИИ

Создать:

```
knowledge/  
  source_registry  
  techniques  
  contraindications  
  evidence_summaries  
  versions
```

Каждая рекомендация должна быть traceable:

```
какая техника  
какой rule  
какие evidence  
какой источник  
какая версия источника
```

ИИ затем только формирует текст под конкретную ситуацию.

Затем — персонализация эффективности перерыва

После каждой рекомендации собирать:

```
accepted?  
snoozed?  
irrelevant?  
completed?  
helped?  
post-break self-report?  
post-break sensor recovery?
```

Можно вычислять:

```
InterventionEffect(person, state, technique)
```

и постепенно персонализировать выбор.

Затем — собственное исследование валидности fatigue model

Это необходимый этап перед сильными коммерческими обещаниями.

Нельзя использовать только ответы самого алгоритма как ground truth.

Понадобится независимая проверка, например:

KSS
Samn-Perelli
PVT в контрольных точках
sensor features
task context
time of day
break history

FAA, например, использует KSS/SP вместе с PVT именно потому, что субъективные и объективные показатели дают разные стороны состояния. ³²

Нужно оценить:

sensitivity
specificity
precision
recall
false alerts/hour
missed high-fatigue episodes
within-person calibration
between-person generalization
calibration drift
performance by glasses/no glasses
performance by lighting

Затем — испытание самого вмешательства

Даже если мы точно предскажем утомление, это ещё не доказывает полезность продукта.

Нужен эксперимент:

A: обычная работа / baseline
B: фиксированный таймер
C: адаптивные рекомендации

Желательно crossover/randomized design.

Измерять:

самооценку fatigue
KSS
PVT / vigilance
task errors
task completion
принятие уведомлений
отклонение уведомлений

false interruptions
perceived annoyance
post-break recovery

Главный бизнес-вопрос:

адаптивная система лучше фиксированного таймера или нет?

Краткие перерывы в среднем способны улучшать субъективное самочувствие, однако метаанализ не обнаружил универсального положительного эффекта microbreak на производительность. Поэтому эффективность именно нашего adaptive intervention engine должна быть доказана собственным экспериментом, а не предположена. ²³

И только затем — банковский production pilot

Перед реальным корпоративным пилотом нужны:

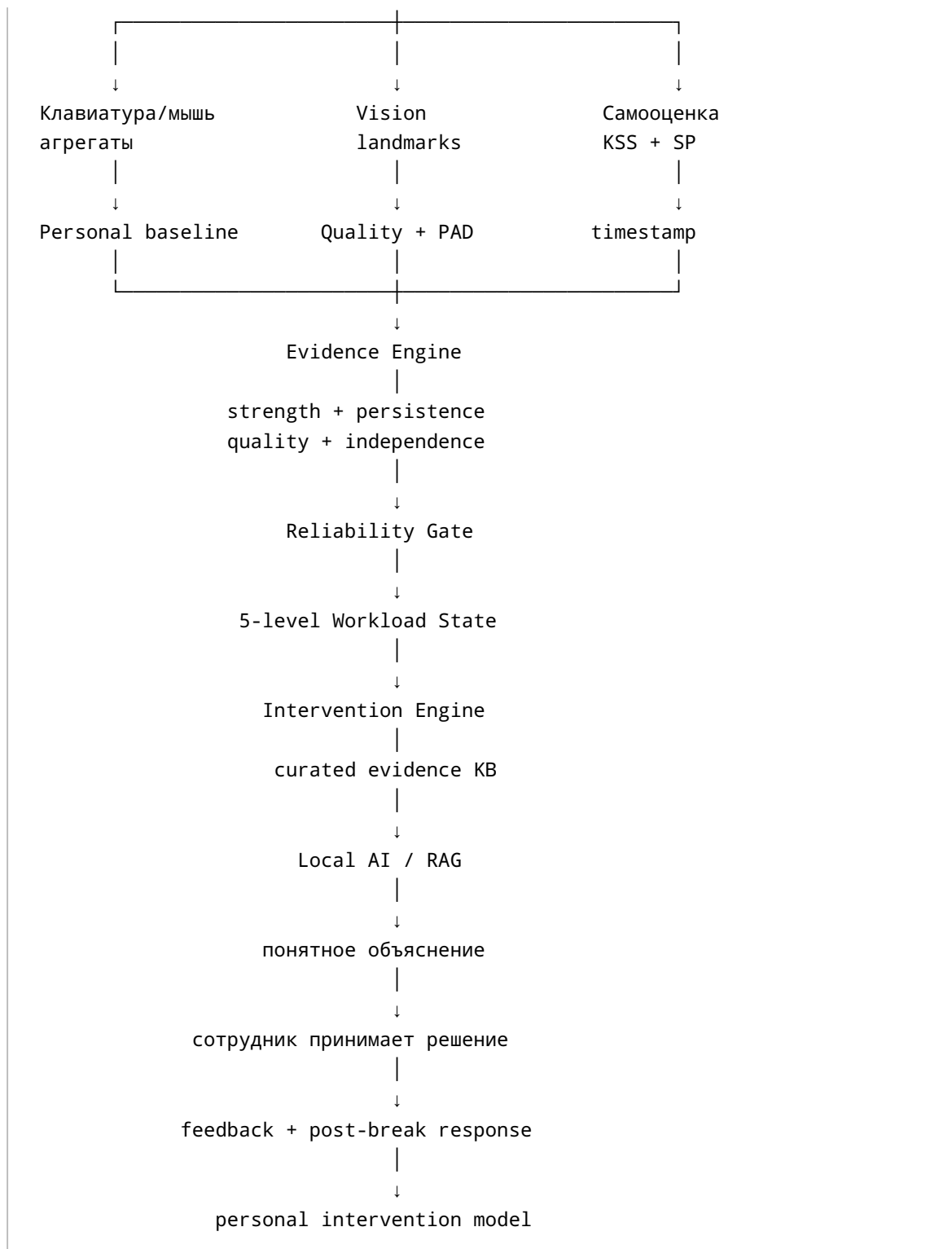
security threat model
privacy impact assessment
legal review
bank IS review
signed binaries
model/component inventory
dependency SBOM
local encryption
secure update
retention policy
rollback
configuration policy
performance budget
accessibility
employee transparency
incident logging
support procedure

Актуальные рекомендации Банка России для систем ИИ финансового рынка прямо делают акцент на threat modelling, минимизации данных, безопасности компонентов, контроле доступа и управлении рисками используемых моделей и open-source технологий. ³³

Каноническая конечная архитектура

Новый разработчик должен держать в голове именно эту схему:

Рабочий контекст
active/passive/...



И несколько правил являются **непредметными для самовольного изменения**:

1. **ACTIVE_WORK** и **PASSIVE_WORK** — не уровни усталости.
2. Никакого **Fatigue 73%** без отдельно доказанной probabilistic model.
3. Vision не включается в решения до размеченной валидации.
4. Сырые видео и ввод не уходят на сервер.
5. Текст клавиатуры не хранится.
6. Самооценка остаётся независимой от сенсорного вывода.

7. KSS и Samn-Perelli имеют отдельные значения и отдельные UX-описания.
8. Пользовательский итог сейчас имеет пять функциональных уровней; надёжность существует отдельно.
9. Main UI содержит три смысловых блока: **Состояние → Основание → Рекомендация**.
10. Технические метрики находятся в **Подробнее**.
11. Камера по умолчанию не показывается крупным видео.
12. Интерфейс — формальный, взрослый и конкретный; без «всё спокойно» и псевдомедицинского «не обнаружено».
13. LLM **не является судьёй усталости**.
14. InterventionEngine принимает решение до LLM.
15. Советы ИИ должны опираться на закрытую экспертную базу.
16. Emotion recognition по лицу не является частью продукта.
17. Anti-spoof и anti-automation определяют **доверие к каналу**, а не «виновность» сотрудника.
18. Порог 45/70/105 минут и нынешние z-thresholds — прототипные эвристики, а не доказанная физиология.
19. Самая ближайшая кодовая задача — **таймер сессии, свежесть данных, исправление recommendation inconsistency и self-report recency**.
20. Главный долгосрочный критерий успеха — не красота детектора, а доказательство того, что **персонализированная своевременная рекомендация восстановления реально полезнее простого фиксированного таймера и не создаёт неприемлемого числа ложных вмешательств**. ³⁴

¹ ² <https://www.cdc.gov/niosh/fatigue/about/index.html>

<https://www.cdc.gov/niosh/fatigue/about/index.html>

³ ⁸ <https://pmc.ncbi.nlm.nih.gov/articles/PMC10108649/>

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10108649/>

⁴ <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX%3A02024R1689-20260727>

<https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX%3A02024R1689-20260727>

⁵ ²⁰ https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

⁶ ¹⁰ <https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0239984>

<https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0239984>

⁷ ²² ³⁴ <https://www.cdc.gov/niosh/bulletin/2022/fatigue-detection-technologies.html>

<https://www.cdc.gov/niosh/bulletin/2022/fatigue-detection-technologies.html>

⁹ <https://pubmed.ncbi.nlm.nih.gov/26517404/>

<https://pubmed.ncbi.nlm.nih.gov/26517404/>

¹¹ <https://www.cdc.gov/niosh/work-hour-training-for-nurses/longhours/mod2/22.html>

<https://www.cdc.gov/niosh/work-hour-training-for-nurses/longhours/mod2/22.html>

¹² <https://www.cdc.gov/niosh/bulletin/2012/sleep-and-work.html>

<https://www.cdc.gov/niosh/bulletin/2012/sleep-and-work.html>

¹³ <https://pubmed.ncbi.nlm.nih.gov/16679057/>

<https://pubmed.ncbi.nlm.nih.gov/16679057/>

¹⁴ ¹⁵ ¹⁶ ³² https://www.faa.gov/data_research/research/med_humanfacs/oamtechreports/media/202615.pdf

https://www.faa.gov/data_research/research/med_humanfacs/oamtechreports/media/202615.pdf

- 17 <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getlastinputinfo>
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getlastinputinfo>
- 18 <https://learn.microsoft.com/en-us/windows/win32/api/winuser/ns-winuser-kbdllhookstruct>
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/ns-winuser-kbdllhookstruct>
- 19 https://docs.opencv.org/4.13.0/dd/d43/tutorial_py_video_display.html
https://docs.opencv.org/4.13.0/dd/d43/tutorial_py_video_display.html
- 21 <https://docs.python.org/3/library/sqlite3.html>
<https://docs.python.org/3/library/sqlite3.html>
- 23 <https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0272460>
<https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0272460>
- 24 <https://www.osha.gov/computer-workstations/hazards-solutions>
<https://www.osha.gov/computer-workstations/hazards-solutions>
- 25 33 <https://www.cbr.ru/Crosscut/LawActs/File/12204>
<https://www.cbr.ru/Crosscut/LawActs/File/12204>
- 26 https://rkn.gov.ru/docs/Raz6jasnenija_RKN_po_biometrii_okonchatel6naja_versija.doc
https://rkn.gov.ru/docs/Raz6jasnenija_RKN_po_biometrii_okonchatel6naja_versija.doc
- 27 <https://www.cbr.ru/crosscut/lawacts/file/9881>
<https://www.cbr.ru/crosscut/lawacts/file/9881>
- 28 <https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8491.pdf>
<https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8491.pdf>
- 29 <https://www.cdc.gov/niosh/ergonomics/ergo-programs/risk-factors.html>
<https://www.cdc.gov/niosh/ergonomics/ergo-programs/risk-factors.html>
- 30 <https://www.cdc.gov/niosh/work-hour-training-for-nurses/longhours/mod5/10.html>
<https://www.cdc.gov/niosh/work-hour-training-for-nurses/longhours/mod5/10.html>
- 31 <https://www.iso.org/obp/ui/en/>
<https://www.iso.org/obp/ui/en/>